



Sistema de Registro y Gestión de Llamadas de Emergencia (SGLE)

Optimización estructurada para la centralización de incidentes en la Región de Tacna.

[STATUS]: Prototipo Funcional v1.0

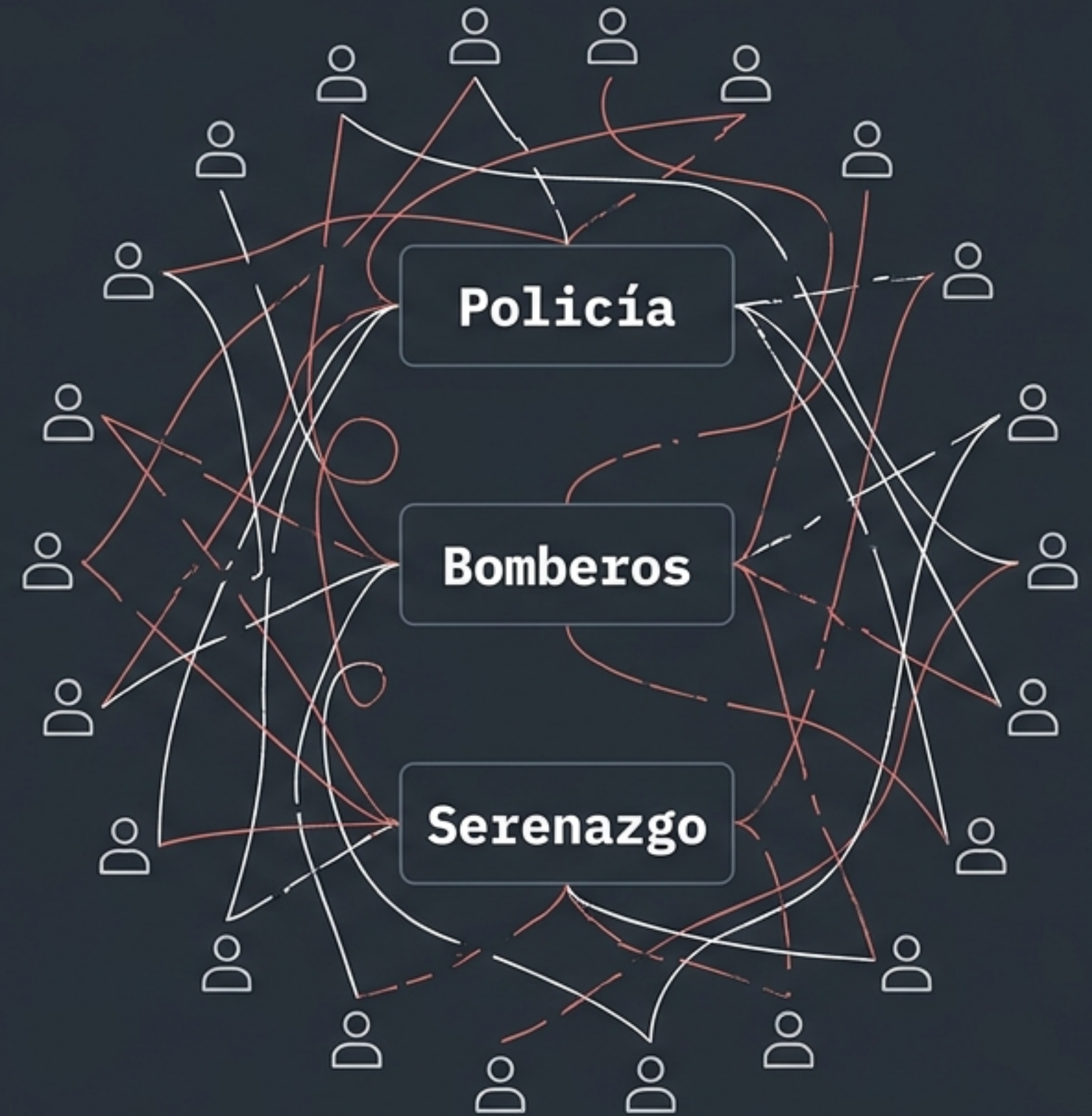
[LANG]: C++ (Estándar ISO)

[TEAM]: Fabia Torres | Alonso Luque | Giovanni Mamani

El Cuello de Botella en Tacna

El crecimiento urbano exige tiempos de respuesta más cortos, pero la descentralización de los reportes genera latencia y pérdida de datos.

- > ERROR: Falta de herramientas informáticas ligeras y unificadas.
- > ERROR: Operadores obligados a registrar incidentes en sistemas aislados.
- > FATAL: Restricciones severas de memoria estática en equipos terminales tradicionales.



Matriz de Diagnóstico: Sistemas Tradicionales vs. SGLE

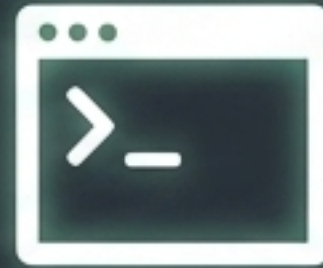
	Sistemas Tradicionales	Propuesta SGLE (C++)
Arquitectura	Alto acoplamiento algorítmico.	Paradigma estructurado y modular.
Rendimiento	Pesado, alta latencia en consola.	Interfaz de baja latencia con bucle do-while continuo.
Gestión de Memoria	Escalabilidad deficiente.	Operaciones dinámicas in-RAM con arreglos compactables.
Mantenimiento	Código monolítico complejo.	Alta cohesión mediante segregación multiarchivo.

Ecosistema Tecnológico Seleccionado



C++ (Estándar ISO)

Elegido por su robustez, eficiencia en la gestión de memoria y soporte nativo para manipulación avanzada de estructuras compuestas.



Dev-C++ (IDE)

Entorno de desarrollo integrado ligero, ideal para la compilación, depuración y enlace del árbol binario en proyectos multiarchivo.



Git & GitHub

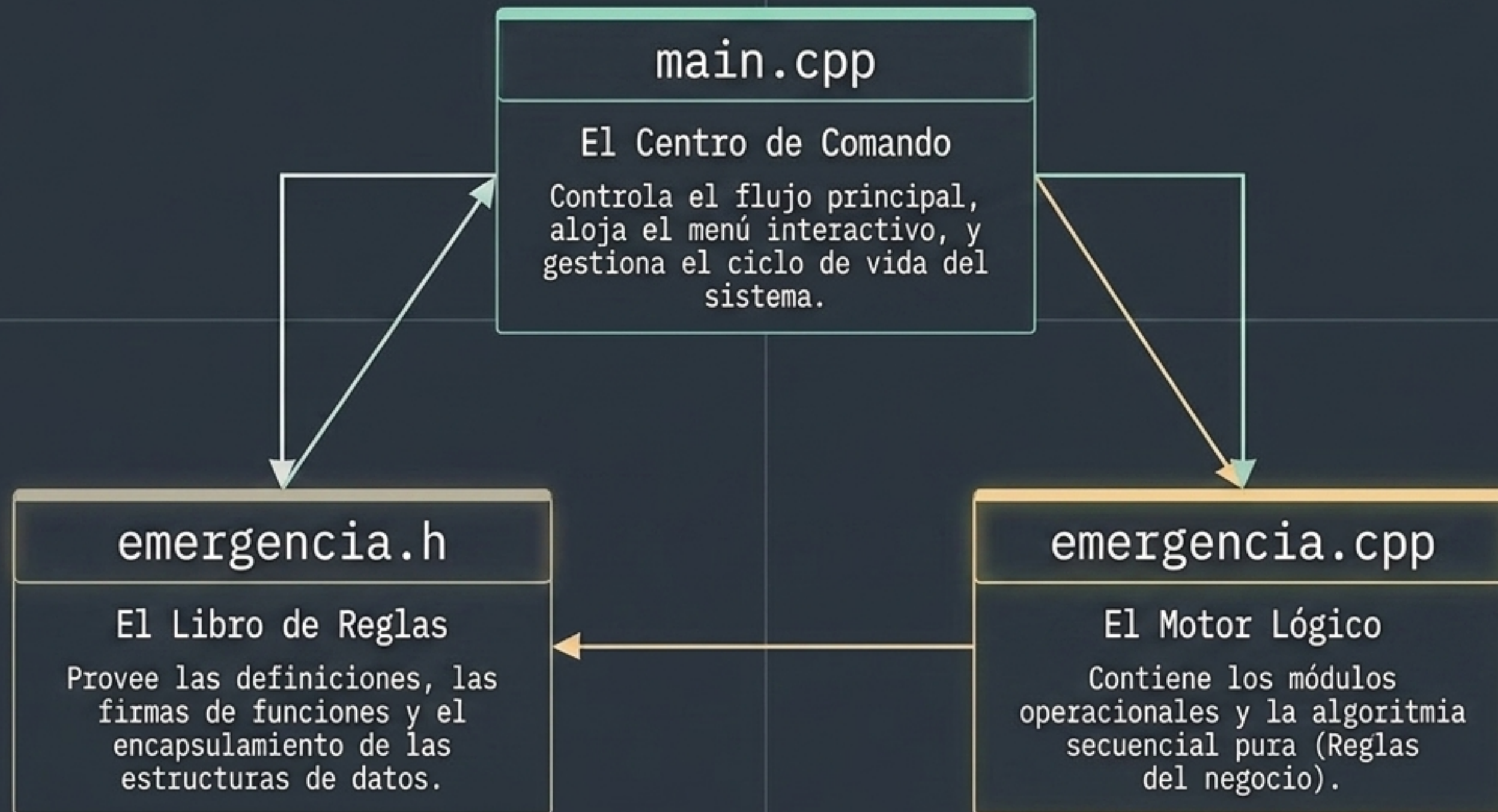
Control de versiones asíncrono para coordinar el trabajo cooperativo, garantizando un historial de commits transparente y auditable.

Abstracción de Datos: struct Emergencia

Digital ID Card		
[int]	idLlamada	: Clave primaria unívoca (indexación).
[string]	nombre	: Identificador del ciudadano reportante.
[string]	zona	: Distrito geográfico del suceso.
[string]	institucion	: Unidad de respuesta (Policía/Bomberos/Serenazgo).
[string]	hora	: Formato temporal 24h (HH:MM).
[string]	estado	: Flag de ciclo de vida (Pendiente / En_Curso / Atendido).

Uso de estructuras de datos homogéneas lineales (arreglos de registros) para persistir datos dinámicamente en RAM.

Arquitectura Multiarchivo (Segregación de Responsabilidades)



Interfaz de Operador: Flujo de Baja Latencia

1. Inicialización:

Se declara el arreglo de registros y el contador en 0.

4. Cierre Seguro:

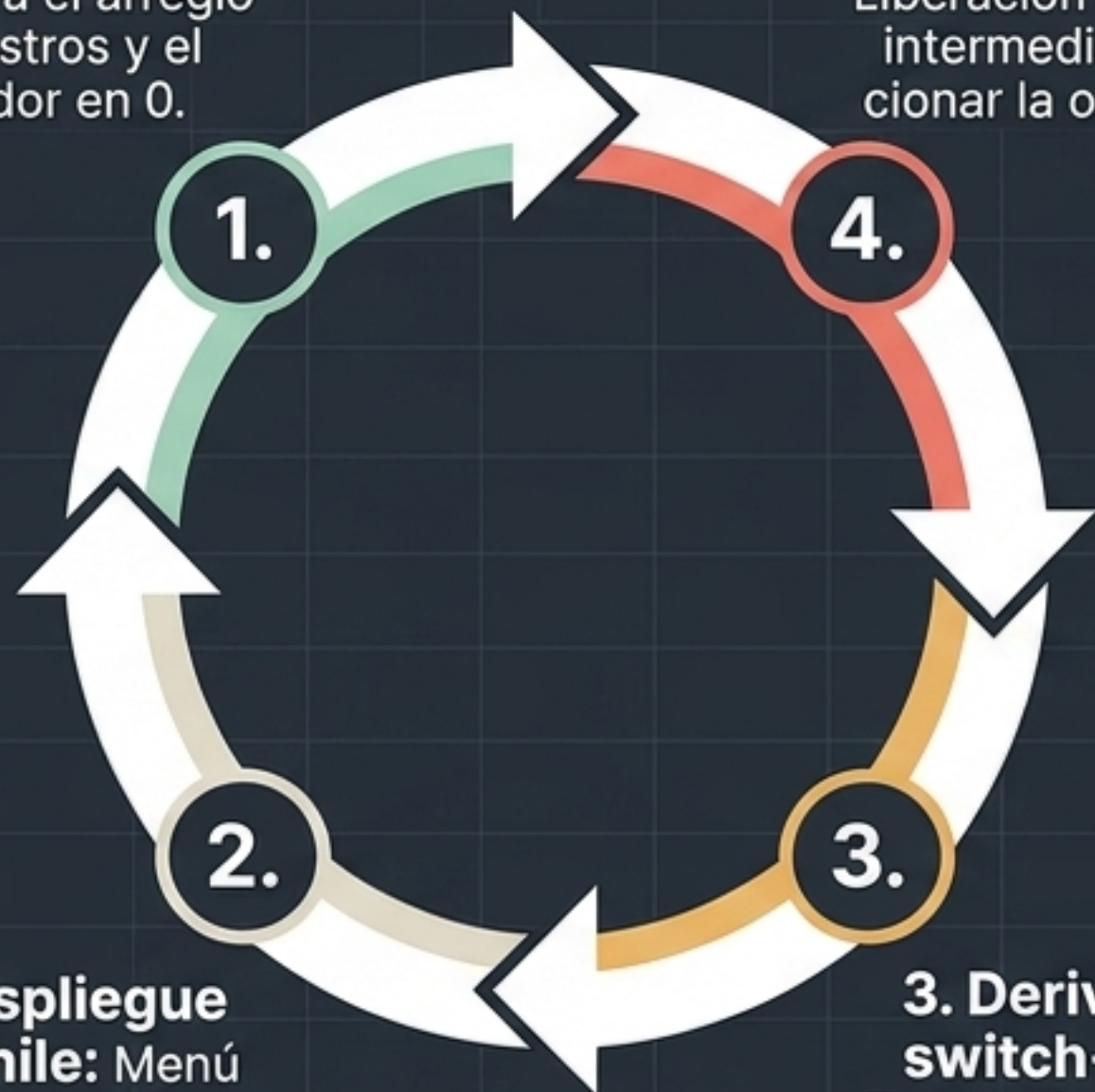
Liberación de memoria intermedia al seleccionar la opción Salir.

2. Despliegue

do-while: Menú interactivo continuo a la espera de inputs.

3. Derivación

switch-case: Enrutamiento hacia las funciones CRUD parametrizadas por referencia.



```
=====
CENTRAL CENTRALIZADA DE EMERGENCIAS - TACNA
=====
1. Registrar Llamada de Emergencia (Create)
2. Listar Todas las Emergencias (Read)
3. Buscar Emergencia por ID (Read)
4. Modificar Estado de la Emergencia (Update)
5. Eliminar Registro de Emergencia (Delete)
6. Salir del Sistema
=====
Seleccione una opcion (1-6):
```


Inserción de Datos (Módulo Create)

```
void registrarLlamada(Emergencia lista[], int &contador) {  
    if (contador >= 100) {  
        cout << "\n[!]! Almacenamiento lleno.\n";  
        lista[contador].idLlamada = 1000 + contador + 1;  
        return;  
    }  
}
```

- ❗ **Validación:** Sistema bloquea inserciones si se superan los 100 registros.
- 🔧 **Automatización:** Generación de clave unívoca (fórmula: $1000 + \text{contador} + 1$).
- 🔧 **Inicialización:** Asignación del estado por defecto a "Pendiente".
- 🔧 **Memoria:** Incremento del contador por referencia (+1).

```
=====
Seleccione una opcion (1-6): 1

--- REGISTRAR EMERGENCIA ---
Nombre del ciudadano (Sin_espacios): FabiaTorres
Zona o Distrito de Tacna: Cercado
Institucion (Policia/Bomberos/Serenazgo): Policia
Hora del suceso (HH:MM): 21:29

[OK] Emergencia guardada con ID: 1001

=====
> █
```


Consulta e Inspección Dinámica (Módulo Read)



Búsqueda por ID (Búsqueda lineal indexada):

```
void buscarEmergencia(const Emergencia lista[], int contador) {  
    if (contador == 0) {  
        cout << "\n[] Sistema vacio.\n";  
        return;  
    }  
    int idBuscado;  
    bool encontrado = false;  
    cout << "\nIngrese el ID de la emergencia a buscar: ";  
    cin >> idBuscado;  
  
    for(int i = 0; i < contador; i++) {  
        if(lista[i].idLlamada == idBuscado) {  
            cout << "\nRegistro Encontrado\n";  
            cout << "ID: " << lista[i].idLlamada << "\nCiudadano: " << lista[i].nombre  
            cout << "\nZona: " << lista[i].zona << "\nUnidad: " << lista[i].institucion  
            cout << "\nHora: " << lista[i].hora << "\nEstado: " << lista[i].estado << endl;  
            encontrade = true;  
            break;  
        }  
    }  
    if(!encontrado) cout << "\n[] No se encontro ninguna llamada con ese ID.\n";  
}
```

```
=====
Seleccione una opcion (1-6): 3
Ingrese el ID de la emergencia a buscar: 1001

[Registro Encontrado]
ID: 1001
Ciudadano: FabiaTorres
Zona: Cercado
Unidad: Policia
Hora: 21:29
Estado: Pendiente
```

Al encontrar la coincidencia exacta con el ID, el sistema interrumpe el bucle (**break**) para ahorrar recursos de procesamiento.

Listar Emergencias (Recorrido iterativo completo):

```
=====
Seleccione una opcion (1-6): 2

--- LISTA GENERAL DE INCIDENTES ---
ID: 1001 | Ciudadano: FabiaTorres | Zona: Cercado | Unidad: Policia | Hora: 21:29 | Estado: Pendiente
```


Actualización Operativa (Módulo Update)



```
void modificarEstado(Emergencia lista[], int contador) {
    if (contador == 0) {
        cout << "\n[!] No hay llamadas para modificar.\n";
        return;
    }
    int idBuscado;
    cout << "\nIngrese el ID de la llamada a actualizar: ";
    cin >> idBuscado;

    for(int i = 0; i < contador; i++) {
        if(lista[i].idLlamada == idBuscado) {
            int opcEstado;
            cout << "\nEstado actual: " << lista[i].estado << endl;
            cout << "Seleccione nuevo estado: 1. En_Curso 2. Atendido\nOpcion: ";
            cin >> opcEstado;
            if(opcEstado == 1) lista[i].estado = "En_Curso";
            else if(opcEstado == 2) lista[i].estado = "Atendido";

            cout << "\n[OK] Estado actualizado correctamente.\n";
            return;
        }
    }
    cout << "\n[!] El ID especificado no existe.\n";
}
```

```
=====
Seleccione una opcion (1-6): 4

Ingrese el ID de la llamada a actualizar: 1001

Estado actual: Pendiente
Seleccione nuevo estado: 1. En_Curso 2. Atendido
Opcion: 2

[OK] Estado actualizado correctamente.
=====
```

El sistema localiza el ID y despliega un submenú que permite al operador conmutar el estado del incidente, sobrescribiendo el atributo en la memoria RAM instantáneamente.

Reestructuración Física de Memoria (Módulo Delete)



ALGORITMO DE ELIMINACIÓN VECTORIAL

```
// 5. ELIMINAR LLAMADA (Delete)
void eliminarLlamada(Emergencia lista[], int &contador) {
    if (contador == 0) {
        cout << "\n[!] Nada que eliminar.\n";
        return;
    }
    int idBuscado;
    cout << "\nIngrese el ID del registro a eliminar: ";
    cin >> idBuscado;

    for(int i = 0; i < contador; i++) {
        if(lista[i].idLlamada == idBuscado) {
            for(int j = i; j < contador - 1; j++) {
                lista[j] = lista[j + 1];
            }
            contador--;
            cout << "\n[OK] Registro eliminado con exito del sistema.\n";
            return;
        }
    }
    cout << "\n[!] No se encontro el ID proporcionado.\n";
}
```

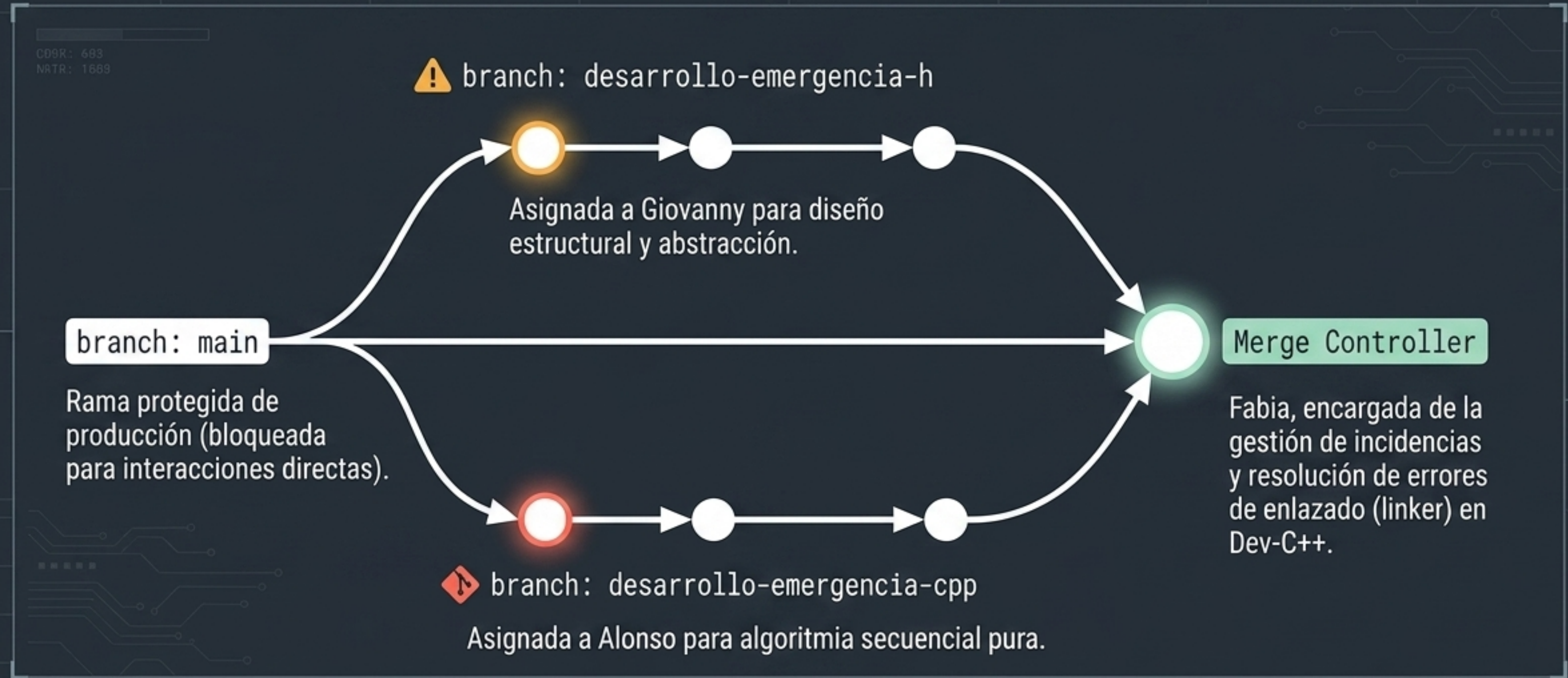
SALIDA DEL SISTEMA (TERMINAL)

```
=====
Seleccione una opcion (1-6): 5

Ingrese el ID del registro a eliminar: 1001

[OK] Registro eliminado con exito del sistema.
=====
```


Metodología de Desarrollo Colaborativo (Git)



Síntesis de Resultados del Proyecto

STATUS: OPTIMIZED

LATENCY: LOW



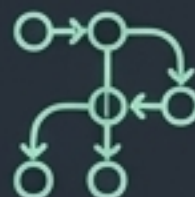
Eficiencia Modular (Técnico)

La segregación en .h y .cpp optimizó la legibilidad del sistema y permitió aislar errores lógicos de manera eficiente, logrando un prototipo ágil y de baja latencia.

LATENCY: LOW

WORKFLOW: ASYNC

BRANCHING: STRICT



Transparencia Asíncrona (Metodológico)

La estrategia de ramificación estricta blindó el código principal contra pérdidas involuntarias, demostrando madurez en el trabajo de laboratorio cooperativo.

BRANCHING: STRICT

CONCEPT: STRUCT

PXTH: POO READY



Pensamiento Abstracto (Académico)

El uso de estructuras compuestas (struct) proveyó al equipo el andamiaje cognitivo indispensable para la futura transición hacia la Programación Orientada a Objetos (POO).

PATH: POO READY

SGLE-Tacna: Sistema Desplegado y Operativo

Solución estructurada, modular y optimizada para la gestión centralizada de incidentes.

- ✓ Ejecución de prototipo finalizada con éxito.
- ✓ Memoria RAM liberada.
- > Sesión cerrada exitosamente.